

COMPLETE HTML & CSS NOTES

Beginner → Intermediate Web Development

Includes: concepts, syntax, examples, common mistakes, layouts, responsive design, Flexbox, Grid, forms, selectors, positioning, animations, and mini projects.

Part	Topics
1	HTML fundamentals, structure, tags, attributes
2	Text, links, images, lists, tables
3	Forms, semantic HTML, multimedia, accessibility
4	CSS fundamentals, selectors, colors, units
5	Box model, display, positioning, z-index
6	Flexbox and CSS Grid
7	Responsive design, media queries
8	Typography, backgrounds, shadows, transitions, animations
9	Best practices + common mistakes
10	Mini projects and revision cheat sheets

1. HTML Fundamentals

HTML stands for HyperText Markup Language. It gives a webpage its structure and meaning. HTML is not a programming language; it is a markup language.

Basic idea: HTML = structure, CSS = appearance, JavaScript = behavior.

Basic HTML document

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My First Page</title>
</head>
<body>
  <h1>Hello World</h1>
  <p>This is my first webpage.</p>
</body>
</html>
```

Important: <head> contains metadata and resources; <body> contains visible page content.

Tags, elements and attributes

A **tag** is syntax such as <p>. An **element** usually includes an opening tag, content, and closing tag. An **attribute** adds information to an element, such as href, src, alt, or class.

```
<a href="https://example.com" target="_blank">Visit</a>

```

Headings and paragraphs

```
<h1>Main heading</h1>
<h2>Section heading</h2>
<h3>Subsection</h3>
<p>This is a paragraph of text.</p>
```

Use headings in a logical hierarchy. Normally a page has one main h1, followed by h2 and h3 sections.

2. HTML Text, Links, Images and Lists

Text formatting

```
<strong>Important text</strong>
<em>Emphasized text</em>
<mark>Highlighted text</mark>
<small>Small text</small>
<del>Deleted text</del>
<ins>Inserted text</ins>
H<sub>2</sub>O
x<sup>2</sup>
```

Links

```
<a href="https://example.com">Open website</a>
<a href="about.html">About page</a>
<a href="#contact">Jump to contact section</a>
<a href="mailto:hello@example.com">Email us</a>
```

For external links, target="_blank" opens a new tab. For security when using it, commonly add rel="noopener noreferrer".

Images

```


<figure>
  
  <figcaption>A snowy mountain.</figcaption>
```

</figure>

alt text is important for accessibility and when an image cannot load.

Lists

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>

<ol>
  <li>Open VS Code</li>
  <li>Create index.html</li>
  <li>Write HTML</li>
</ol>

<dl>
  <dt>HTML</dt>
  <dd>Structures a webpage.</dd>
</dl>
```

3. HTML Tables, Forms and Semantic HTML

Tables

```
<table>
  <caption>Student Marks</caption>
  <thead>
    <tr><th>Name</th><th>Maths</th></tr>
  </thead>
  <tbody>
    <tr><td>Rahul</td><td>90</td></tr>
    <tr><td>Aman</td><td>85</td></tr>
  </tbody>
</table>
```

<tr> = row, <th> = header cell, <td> = data cell. Tables are for tabular data, not for page layout.

Forms

```
<form action="/submit" method="post">
  <label for="name">Name:</label>
  <input id="name" name="name" type="text" required>

  <label for="email">Email:</label>
  <input id="email" name="email" type="email" required>

  <label for="course">Course:</label>
  <select id="course" name="course">
    <option value="cs">Computer Science</option>
    <option value="ai">AI</option>
  </select>

  <label for="message">Message:</label>
  <textarea id="message" name="message"></textarea>

  <button type="submit">Submit</button>
</form>
```

Useful input types: text, email, password, number, date, radio, checkbox, file, submit.

Semantic HTML

```
<header>Website header</header>
<nav>Navigation links</nav>
<main>
  <section>
    <h2>Section</h2>
    <article>Independent content</article>
  </section>
  <aside>Related information</aside>
</main>
<footer>Copyright information</footer>
```

Semantic tags communicate the role of content to browsers, search engines, and assistive technologies.

Multimedia

```
<audio controls src="song.mp3"></audio>

<video controls width="500">
  <source src="video.mp4" type="video/mp4">
</video>

<iframe src="https://example.com" title="Embedded page"></iframe>
```

4. CSS Fundamentals

CSS stands for Cascading Style Sheets. It controls colors, spacing, typography, layout, responsiveness, and visual effects.

Three ways to add CSS

```
<!-- Inline -->
<p style="color: blue;">Hello</p>

<!-- Internal -->
<style>
  p { color: blue; }
</style>

<!-- External (recommended) -->
<link rel="stylesheet" href="style.css">
```

CSS syntax

```
selector {
  property: value;
}

p {
  color: blue;
  font-size: 18px;
}
```

The **selector** chooses elements. A **property** is what you change. A **value** specifies how it should look.

Selectors

```
/* Element */
p { color: black; }

/* Class */
.card { padding: 20px; }

/* ID */
#header { height: 80px; }

/* Multiple */
h1, h2 { font-family: Arial; }

/* Descendant */
nav a { text-decoration: none; }

/* Child */
nav > a { color: red; }

/* Attribute */
input[type="email"] { border: 1px solid gray; }

/* Pseudo-class */
button:hover { transform: scale(1.03); }
```

Class can be reused on many elements. **ID** should normally identify one unique element on a page. In CSS specificity, ID selectors are stronger than class selectors, which are stronger than element selectors.

5. CSS Colors, Units, Typography and Box Model

Colors

```
color: red;
color: #2563eb;
color: rgb(37, 99, 235);
color: rgba(37, 99, 235, 0.5);
background-color: #f5f5f5;
```

Common units

px = fixed CSS pixel unit. **%** = relative to a relevant parent/container. **em** = relative to the current element's font size. **rem** = relative to the root font size. **vw/vh** = viewport width/height. Use rem, %, and viewport units where appropriate for flexible layouts.

Typography

```
body {
  font-family: Arial, sans-serif;
  font-size: 16px;
  line-height: 1.6;
}

h1 {
  font-size: 2rem;
  font-weight: 700;
  text-align: center;
}

a {
  text-decoration: none;
}
```

The Box Model

Every normal HTML element can be thought of as a box: **content** → **padding** → **border** → **margin**. Padding is inside the border; margin is outside the border.

```
* {
  box-sizing: border-box;
}

.card {
  width: 300px;
  padding: 20px;
  border: 2px solid black;
  margin: 30px;
}
```

Best practice: box-sizing: border-box makes width/height calculations easier because padding and border are included in the declared size.

6. Display, Positioning and Z-Index

Display

```
display: block;
display: inline;
display: inline-block;
display: none;
display: flex;
display: grid;
```

block: starts on a new line and generally takes available width. **inline:** stays within a line. **inline-block:** stays inline but accepts width/height. **none:** removes the element from layout.

Position

```
position: static; /* default */
```

```
position: relative;
position: absolute;
position: fixed;
position: sticky;
```

relative keeps the element in normal flow but allows offsets. **absolute** positions an element relative to its nearest positioned ancestor. **fixed** is attached to the viewport. **sticky** behaves normally until a scroll threshold is reached.

```
.parent {
  position: relative;
}

.child {
  position: absolute;
  top: 10px;
  right: 10px;
}

.chat-button {
  position: fixed;
  right: 20px;
  bottom: 20px;
}
```

Z-index

z-index controls stacking order when elements overlap. It works in contexts where positioning/stacking rules allow it.

```
.menu {
  position: absolute;
  z-index: 100;
}
```

7. Flexbox — One-Dimensional Layout

Flexbox is ideal for arranging items in a row or column.

```
.container {
  display: flex;
  flex-direction: row;
  justify-content: space-between;
  align-items: center;
  gap: 20px;
  flex-wrap: wrap;
}
```

Important Flexbox properties

Parent/container: display, flex-direction, justify-content, align-items, align-content, gap, flex-wrap.

Child/item: flex, flex-grow, flex-shrink, flex-basis, align-self, order.

Example: centered button

```
.hero {
  min-height: 300px;
  display: flex;
  justify-content: center;
  align-items: center;
}
```

Example: responsive cards

```
.cards {
  display: flex;
  flex-wrap: wrap;
  gap: 16px;
}

.card {
  flex: 1 1 250px;
}
```

8. CSS Grid — Two-Dimensional Layout

CSS Grid is excellent for layouts involving both rows and columns.

```
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}
```

Useful Grid properties

grid-template-columns, grid-template-rows, gap, grid-column, grid-row, place-items, align-items, justify-items.

Responsive grid

```
.products {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 20px;
}
```

This pattern automatically fits as many columns as the available width allows while keeping cards at a sensible minimum width.

9. Responsive Design

Responsive design means the webpage adapts to different screen sizes such as phones, tablets, and desktops.

Viewport meta tag

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Media queries

```
/* Desktop-first example */
.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}

/* On smaller screens */
@media (max-width: 768px) {
  .container {
    grid-template-columns: 1fr;
  }
}
```

A good responsive site avoids fixed widths that cause horizontal scrolling. Prefer flexible widths, max-width, Grid/Flexbox, and responsive typography.

10. Backgrounds, Borders, Shadows and Effects

Backgrounds

```
.hero {
  background-color: #111827;
  background-image: url("hero.jpg");
  background-size: cover;
  background-position: center;
}
```

Borders and radius

```
.card {
  border: 1px solid #ddd;
  border-radius: 16px;
}
```

Shadows

```
.card {  
  box-shadow: 0 8px 25px rgba(0,0,0,0.12);  
}
```

Opacity and overflow

```
.muted {  
  opacity: 0.7;  
}  
  
.box {  
  overflow: hidden;  
}
```

11. Transitions and Animations

Transitions

```
.button {  
  background: black;  
  color: white;  
  transition: transform 0.2s ease, opacity 0.2s ease;  
}  
  
.button:hover {  
  transform: translateY(-2px);  
  opacity: 0.9;  
}
```

Keyframe animation

```
@keyframes slideIn {  
  from {  
    transform: translateX(-30px);  
    opacity: 0;  
  }  
  to {  
    transform: translateX(0);  
    opacity: 1;  
  }  
}  
  
.title {  
  animation: slideIn 0.6s ease;  
}
```

Use animations purposefully. Too much motion can make a page distracting. Respect users who prefer reduced motion when appropriate.

```
@media (prefers-reduced-motion: reduce) {  
  *, *::before, *::after {  
    animation-duration: 0.01ms !important;  
    transition-duration: 0.01ms !important;  
  }  
}
```

12. CSS Variables and Reusable Design

CSS custom properties let you store reusable values.

```
:root {  
  --primary: #2563eb;  
  --text: #111827;  
  --surface: #ffffff;  
  --radius: 12px;  
}  
  
.button {  
  background: var(--primary);  
  color: white;  
  border-radius: var(--radius);  
}
```

```
.card {
  color: var(--text);
  background: var(--surface);
}
```

This makes large projects easier to maintain because changing one variable can update many components.

13. Complete Example — Simple Portfolio

The following example combines semantic HTML, an external stylesheet, Flexbox, Grid, forms, and responsive design.

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Siddhant | Portfolio</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<header class="site-header">
  <nav class="navbar">
    <a class="logo" href="#">Siddhant</a>
    <div class="nav-links">
      <a href="#about">About</a>
      <a href="#projects">Projects</a>
      <a href="#contact">Contact</a>
    </div>
  </nav>
</header>

<main>
  <section class="hero">
    <div>
      <p class="eyebrow">CS & AI Student</p>
      <h1>Building useful things with technology.</h1>
      <p>I am learning web development and software engineering.</p>
      <a class="button" href="#projects">View Projects</a>
    </div>
  </section>

  <section id="about" class="section">
    <h2>About Me</h2>
    <p>I enjoy learning programming and building projects.</p>
  </section>

  <section id="projects" class="section">
    <h2>Projects</h2>
    <div class="projects">
      <article class="card">
        <h3>Project One</h3>
        <p>A simple HTML and CSS project.</p>
      </article>
      <article class="card">
        <h3>Project Two</h3>
        <p>A responsive website.</p>
      </article>
      <article class="card">
        <h3>Project Three</h3>
        <p>A JavaScript project.</p>
      </article>
    </div>
  </section>

  <section id="contact" class="section">
    <h2>Contact</h2>
    <form class="contact-form">
      <label for="name">Name</label>
      <input id="name" type="text" required>

      <label for="email">Email</label>
      <input id="email" type="email" required>

      <label for="message">Message</label>
      <textarea id="message" rows="5"></textarea>
    </form>
  </section>
</main>
</body>
</html>
```

```

        <button class="button" type="submit">Send</button>
    </form>
</section>
</main>

<footer>
    <p>© 2026 Siddhant</p>
</footer>

</body>
</html>

```

style.css

```

:root {
    --primary: #2563eb;
    --text: #111827;
    --surface: #ffffff;
    --background: #f8fafc;
}

* {
    box-sizing: border-box;
}

body {
    margin: 0;
    font-family: Arial, sans-serif;
    color: var(--text);
    background: var(--background);
    line-height: 1.6;
}

.navbar {
    max-width: 1100px;
    margin: auto;
    padding: 16px 20px;
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.nav-links {
    display: flex;
    gap: 20px;
}

a {
    color: inherit;
    text-decoration: none;
}

.hero {
    max-width: 1100px;
    min-height: 500px;
    margin: auto;
    padding: 80px 20px;
    display: flex;
    align-items: center;
}

.hero h1 {
    max-width: 700px;
    font-size: clamp(2.2rem, 6vw, 4.5rem);
    line-height: 1.05;
}

.button {
    display: inline-block;
    padding: 12px 18px;
    margin-top: 10px;
    background: var(--primary);
    color: white;
    border-radius: 10px;
}

.section {
    max-width: 1100px;
    margin: auto;
    padding: 70px 20px;
}

```

```

.projects {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}

.card {
  padding: 24px;
  background: var(--surface);
  border-radius: 14px;
  box-shadow: 0 8px 25px rgba(0,0,0,0.08);
}

.contact-form {
  max-width: 600px;
  display: grid;
  gap: 10px;
}

input, textarea {
  width: 100%;
  padding: 12px;
  border: 1px solid #ccc;
  border-radius: 8px;
  font: inherit;
}

footer {
  text-align: center;
  padding: 30px;
}

@media (max-width: 768px) {
  .nav-links {
    gap: 10px;
    font-size: 14px;
  }

  .projects {
    grid-template-columns: 1fr;
  }

  .hero {
    min-height: 400px;
  }
}

```

14. Common Mistakes to Avoid

Mistake	Better approach
Using for page spacing	Use CSS margin/padding.
Using tables for layout	Use Flexbox or Grid.
Forgetting alt text	Add meaningful alt text to informative images.
Too many IDs for styling	Prefer reusable classes.
Fixed width everywhere	Use %, max-width, Grid/Flexbox, and responsive rules.
Putting CSS inline everywhere	Use an external stylesheet for maintainability.
Huge unoptimized images	Resize/compress images appropriately.
Ignoring mobile screens	Test responsive layouts.
Using div for everything	Use semantic HTML where it fits.

15. HTML + CSS Quick Revision Sheet

HTML essentials

html root • head metadata • body visible content • h1-h6 headings • p paragraph • a link • img image • ul/ol/li lists • table data tables • form/input/button forms • header/nav/main/section/article/aside/footer semantic structure.

CSS essentials

Selector → property → value. Know: color, background, font, margin, padding, border, width, height, display, position, z-index, flex, grid, gap, transition, animation, @media.

Layout decision rule

Need simple row/column alignment? → Flexbox. **Need rows + columns?** → Grid. **Need an element attached to a corner/viewport?** → Positioning. **Need different layouts for screen sizes?** → Media queries + flexible sizing.

Practice order

1. Build a basic HTML page → 2. Style text and colors → 3. Learn box model → 4. Build navbar → 5. Learn Flexbox → 6. Learn Grid → 7. Make it responsive → 8. Build forms/cards → 9. Add transitions → 10. Build complete projects.

Mini-project ideas

Personal portfolio • Landing page • Product card page • Blog layout • Restaurant page • College profile page • To-do UI • Pricing page • Responsive dashboard • Your own LIC informational website.

Final advice: Don't just read these notes. Type every example yourself, change values, break the code intentionally, and fix it. That is how HTML/CSS actually sticks.